

Spec 01 — Modelo de dados & contratos

Assume PostgreSQL (ver ADR 0002). `snake_case` no banco → `camelCase` na resposta JSON. Os tipos espelham `src/entities/*/model.ts` e os contratos de `src/mocks/handlers.ts`.

1. Schema

```

create table events (
  id          uuid          primary key default gen_random_uuid(),
  host_id     uuid          not null references auth.users(id) on delete cascade,
  title       text          not null check (char_length(title) between 1 and 120),
  birth_date  date          not null,
  theme       text          not null default '',
  message     text          not null default '',
  cover_url   text          not null default '',
  list_token  text          not null unique default encode(gen_random_bytes(16),'hex'), -- 128 bits
  surprise_mode boolean     not null default false,
  status      text          not null default 'draft' check (status in ('draft','active','past')),
  created_at  timestampz    not null default now()
);
create index events_host_id_idx on events (host_id);

create table gifts (
  id          uuid          primary key default gen_random_uuid(),
  event_id    uuid          not null references events(id) on delete cascade,
  name        text          not null check (char_length(name) between 1 and 200),
  description  text          not null default '',
  image_url   text          not null default '',
  reference_price numeric(10,2) check (reference_price >= 0),
  store_link   text          not null default '',
  link_meta   jsonb,         -- Open Graph cacheado
  most_wanted  boolean     not null default false,
  is_group     boolean     not null default false,
  group_target numeric(10,2), -- obrigatório se is_group
  status      text          not null default 'available' check (status in ('available','reserved')),
  created_at  timestampz    not null default now(),
  constraint group_target_required check (not is_group or group_target is not null)
);
create index gifts_event_id_idx on gifts (event_id);

create table reservations (
  id          uuid          primary key default gen_random_uuid(),
  gift_id     uuid          not null references gifts(id) on delete cascade,
  event_id    uuid          not null references events(id) on delete cascade,
  guest_name  text          not null check (char_length(guest_name) between 1 and 100),
  guest_email text,         -- opcional; privado por RLS
  message     text          not null default '',
  idempotency_key text      not null unique,         -- UUID v4 do cliente, 1 por form
  contribution numeric(10,2), -- presente em grupo
  slot        int          not null default 1 check (slot >= 1),
  status      text          not null default 'active' check (status in ('active','cancelled')),
  created_at  timestampz    not null default now(),
  cancelled_at timestampz
);
create index reservations_gift_id_idx on reservations (gift_id);
-- invariantes de reserva única: ver Spec 02

create table reservation_log (      -- auditoria leve, imutável
  id          uuid          primary key default gen_random_uuid(),
  reservation_id uuid      not null references reservations(id),
  action      text          not null check (action in ('created','cancelled')),
  actor_name  text          not null,
  occurred_at timestampz    not null default now()
);

create table guests (
  id          uuid          primary key default gen_random_uuid(),
  event_id    uuid          not null references events(id) on delete cascade,
  name        text          not null,
  email       citext        not null check (email ~* '^[^@]+@[^@]+\.[^@]+$',) -- PII: protegida por RLS
  created_at  timestampz    not null default now(),
  unique (event_id, email)
);

create table rsvps (

```

```

    id          uuid          primary key default gen_random_uuid(),
    event_id    uuid          not null references events(id) on delete cascade,
    name        text          not null check (char_length(name) between 1 and 100),
    email       text,
    status      text          not null default 'confirmed' check (status in ('confirmed','declined')),
    created_at  timestamptz not null default now()
);

create table messages (      -- mural de recados
    id          uuid          primary key default gen_random_uuid(),
    event_id    uuid          not null references events(id) on delete cascade,
    author      text          not null check (char_length(author) between 1 and 100),
    text        text          not null check (char_length(text) between 1 and 500),
    created_at  timestamptz not null default now()
);

create table url_enrichment_cache ( -- ver Spec 04
    url         text          primary key,
    title       text,
    image_url   text,
    price       numeric(10,2),
    fetched_at  timestamptz not null default now()
);

create table notification_log ( -- idempotência de e-mail; ver Spec 04
    id          uuid          primary key default gen_random_uuid(),
    event_type   text          not null,
    reference_id text          not null,
    sent_at     timestamptz not null default now(),
    unique (event_type, reference_id)
);

```

2. Estados

Presente (**status** é **derivado** das reservas, nunca fonte de verdade solta):

- comum: **available** → **reserved** (1 reserva ativa) → **available** (cancelou)
- grupo: permanece **available** enquanto **SUM(contribution ativa) < group_target** ; vira **reserved** ao atingir a meta

Reserva: nasce **active** ou não nasce (a transação falha — nunca há estado intermediário); **active** → **cancelled** .

3. DTOs principais

```

// GET /api/registry/:token
interface RegistryResponse {
  event: { id; hostId; title; birthDate; theme; message; coverUrl; listToken; surpriseMode };
  host: { id; name };
  gifts: Array<{
    id; eventId; name; description; imageUrl;
    referencePrice: number | null; storeLink; mostWanted; isGroup;
    status: "available" | "reserved";
    groupGoal?: { target: number; collected: number }; // só quando isGroup
  }>;
  // nunca inclui guests[]; nunca inclui quem reservou quando surpriseMode
}

// POST /api/gifts/:id/reservation
interface ReserveGiftRequest {
  guestName: string;          // 1-100
  guestEmail?: string;
  message: string;            // 0-500
  idempotencyKey: string;      // UUID v4 (crypto.randomUUID) por form
  contribution?: number;       // obrigatório se isGroup
}

interface ReserveGiftResponse { reservation: { id; giftId; guestName; message; idempotencyKey; createdAt } }
// 409 { code: "ALREADY_RESERVED" } | 422 { code: "VALIDATION_ERROR" }

```

Demais DTOs (criar/editar evento e presente, RSVP, recado, convidado, enrich-link) seguem o mesmo formato dos handlers MSW atuais.